UNIVERSITÉ ÉVRY
PARIS-SACLAY

OOP Project Report:

Modelling and Implementing a Machine Learning Library

M1 GENIOMHE

2025 – 2026

Name:	Surname:	Student ID:
Boomika	*SELVARADJOU*	*20220362*
Sitty Nouzlat	*SAID CHANFI*	*20235430*
Mah	*NOOR*	*20257462*

Table of contents:

I) Introduction
II) Our approach to model the library
III) Detailed Code Analysis and Evolution of the Solution
IV) Evaluation of the Library and Comparative Analysis with PyTorch
V) Limitations of the Model and Implementation
VI) Conclusion

I) Introduction:

This goal of this project is to design a user-friendly Python machine-learning (*ML*) library, built using Object-Oriented Programming (*OOP*) principles and advanced Python features such as decorators, proprieties, class methods and dunder methods. This project requires a rigorous application of both mathematical understanding and solid software architecture because it carefully avoids the usage of existing ML implementations and relies solely on *NumPy*. The library enables users to load and preprocess datasets, train ML models and evaluate their performance through a user-friendly interface. This highlights how an adaptable and well-organized object-oriented design is used to execute the entire ML workflow, including data preprocessing, model training, evaluation and testing.

This report retraces the complete development of library *upsaclay_ml*, from its architectural design to final evaluation. To guarantee interoperability across components like *Data Loaders*, *Transformers*, and *Models*, we first examine how advanced OOP concepts like encapsulation and deep inheritance hierarchies were applied. Next, we detail how these mathematical models were translated into reliable Python code. We conclude by validating the library using functional testing and performance benchmarks, to show that creating a library from scratch offers valuable insights to better grasp the balance between code simplicity, organisation and performance.

II) Our approach to model the library:

- **Class diagram:**

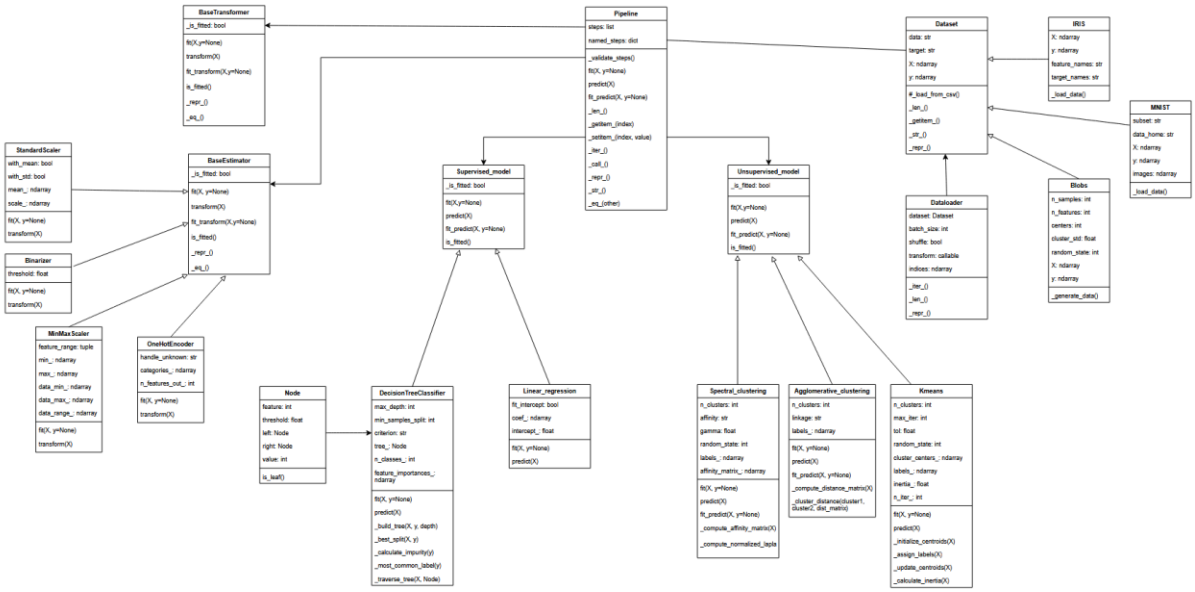


Figure 1.a: Class diagram and overview of the main classes and their relationships in the library (final version).

The class diagram (*Figure 1.a*) of our library is structured around two base classes, *BaseEstimator* and *BaseTransformer,* which are for data preprocessing and modeling

components. *BaseTransformer* requires implementations of fit and transform for the data transformations steps, with *fit_transform* and fitted-state tracking. Transformers like *StandardScalers*, *MinMaxScaler*, *Binarizer* and *OneHotEncoder* inherit from this and are extended with specific parameters and attributes.

BaseEstimator defines the interface for predictive models through *fit*, *predict* methods, *fit_predict*, equality comparisons and parameter introspection. Estimators are needed for learning from transformed data and producing predictions.

Pipeline class is the coordinating component that composes multiple transformers followed by the final estimator. It maintains the ordered steps of transformations, with the last step being an estimator. During execution, the pipeline sequentially applies fit and transform operations on transformers before getting to the estimator.

We have data handling components like *Dataset* and *DataLoader*. The *Dataset* class represents collection of data and targets, and a unified interface to access samples. Datasets like *IRIS*, *MNISR* & *Blobs* inherit from *Dataset*. The *DataLoader* class is associated with *Dataset* by composition, managing batching, shuffling and iteration over samples for training and evaluation.

For modeling, we have classes for supervised and unsupervised learning. A *Node* class is the building block of tree-based models, storing split conditions, child references and prediction values. The *DecisionTree* class is defined by multiple *Node* objects and is the overall tree structure. Supervised decision trees inherit from *BaseEstimator* and rely on labelled data, while unsupervised variants have the same structure for tasks as clustering, reusing through inheritance.

Overall, the class diagram emphasizes the architecture of our library, with inheritance for common behaviours and composition used to create complex workflow. Effectively, capturing the relationships between its components and designed to support extensibility.

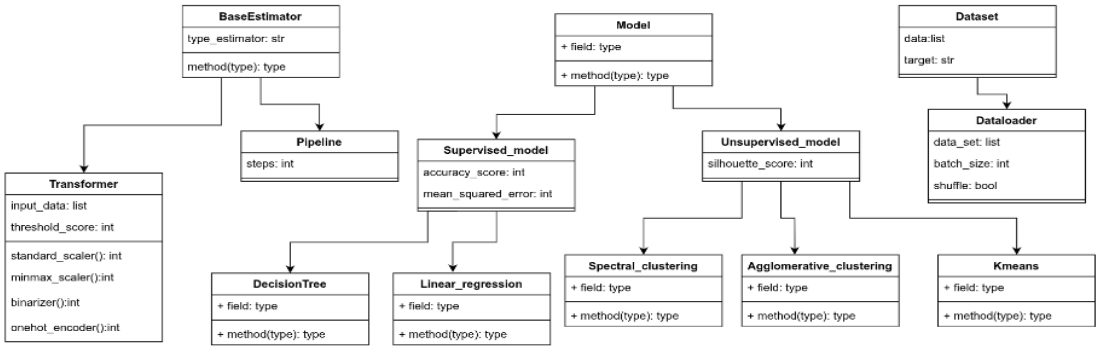


Figure 1.b: Class diagram and overview of the main classes and their relationships in the library (first version).

The class diagram presented in *Figure 1.b* illustrates the initial architectural design of our library. This preliminary structure establishes the fundamental separation between

preprocessing (*Transformer*) and modelling (*BaseEstimator*). It introduces the *Pipeline* as a central coordinator to link these two branches and creates a clear distinction between Supervised and Unsupervised models. Furthermore, it explicitly models data management through the *Dataset* and *Dataloader* classes to standardize input handling from the very beginning.

The main evolutions in our class diagrams are the addition of new classes and changes to the base classes. We also developed our model by adding a dataset and a data loader. In addition, we optimized the associations to achieve a more structured and well-organized implementation.

- **Principles of approach:**

The library implements both supervised and unsupervised models. For supervised learning, we implement a *DecisionTreeClassifier*, which requires labelled target data and evaluates performance by comparing predictions against truth-labels.

For unsupervised learning, we include K-means, Agglomerative and Spectral clustering, where models assign observations to clusters without prior labels.

For data handling, the library allows users to load datasets in csv format and provides a data loader that provides the data in mini batches. Before model training, the dataset undergoes preprocessing transformations, like *StandardScaler*, *MinMaxScaler*, *Binarizer* and *OneHotEncoder*, which ensures that the data is suitable for the training.

For efficiency and to be user-friendly, the library has a pipeline mechanism that enables different preprocessing steps and models them into one singular optimal workflow. Also, model selection utilities are given, such as a train-test split function to separate datasets for training and evaluation.

Our model can be evaluated for its performance with metrics such as, accuracy scores or mean squared error for supervised models, and silhouette score for unsupervised clustering models. To ensure reliability and code quality, the project is integrated with a continuous integration pipeline in GitHub, which lets us automatically test our model in order to improve our implementation further.

III) Detailed Code Analysis and Evolution of the Solution

1. Dataset:

- **Overview:**

The *Dataset* class helps load, store and give access to data. The design goal is to make our library work similarly to *scikit-learn*, in a user-friendly manner. The input data (*features*) are stored in an attribute *X*, and for supervised datasets, the target values (*labels*) are stored in *y*.

Standard methods (*__getitem__*, *__len__*, *__str__*, *__repr__*) ensure datasets are compatible with models, transformers, and the data loader.

The dataset module has three main parts:

- Base Dataset class: Supports *CSV/URL* loading or direct array, and defines common behaviour for all datasets

- Built-in datasets: *MNIST*, *Iris*, *Blobs* are datasets provided for experimentation and testing.

- DataLoader: handles batching, shuffling and mini-batch iteration.

```
class Dataset:
    def __init__(self, X=None, y=None, path=None):
        if path is not None:
            df = pd.read_csv(path)
            self.X = df.iloc[:, :-1].values
            self.y = df.iloc[:, -1].values
        else:
            self.X = np.asarray(X)
            self.y = None if y is None else np.asarray(y)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        if self.y is None:
            return self.X[idx]
        return self.X[idx], self.y[idx]
```

- **Design choices & iterations:**

The base *Dataset* class was kept simple rather than abstract to support direct *CSV/URL* loading. External machine learning libraries were avoided to comply with project restrictions. The Iris dataset was embedded to remove dependency on remote resources, while *MNIST* downloading was preferred over shipping binary files to keep the repository clean. The *__str__()* method was extended to show class statistics for more informative output.

- **Limitations & future Improvements:**

The current implementation relies on external servers to download the *MNIST* dataset, doesn't provide built-in support for train/test splitting, and only accepts numeric *CSV* input, requiring manual preprocessing for categorical data.
Potential improvements could include caching with hashing, a dedicated *from_url()* helper function, automatic detection for mixed data types, and closer integration with transformers to allow lazy or on-demand loading.

2. Transformers

- **Overview:**

The transformer module provides basic preprocessing operations before training models, in order to make transformations consistent and compatible with models and pipelines. All transformers inherit from *BaseTransformer* and follow these sequential transformations, a *fit* phase (to compute statistics), a *transform* phase (to apply changes), and then *fit_transform*.

The implementations cover scaling (*StandardScaler, MinMaxScaler*), normalization, thresholding and categorical encoding (*Binarizer, and OneHotEncoder*).

```
class BaseTransformer:
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        raise NotImplementedError()

    def fit_transform(self, X, y=None):
        self.fit(X, y)
        return self.transform(X)
```

- **Module structure:**

The *__init__.py* file exposes the public transformers, enabling clean imports and the implementations are contained in *preprocessing.py* file. This structure is minimal, clear, and designed to be easily extended without breaking the user-facing code.

- **Transformers details:**

StandardScaler standardizes numeric features by removing the mean and dividing by the standard deviation. *MinMaxScaler* scales features into a fixed range, with the default range of [0, 1]. *Binarizer* converts numerical values into 0 or 1 based on a specific threshold and is stateless besides marking itself as fitted. *OneHotEncoder* converts integer categorical values into binary indicator vectors and can handle unknown categories based on configuration.

- **Design choices:**

The main design choice was enforcing a unified API (*fit, transform, fit_transform*) so transformers integrate smoothly into pipelines. Safety checks, such as protection against division by zero, were added early. The module avoids external ML packages and stays modular to facilitate future extensions.

- **Limitations & future improvements:**

Currently, transformers assume numeric or integer inputs, so handling strings or mixed type data requires manual preprocessing. Inverse transformations are not yet supported. Future improvements could add normalization, imputation, discretization, support for string category, and the ability to perform inverse transformations.

3. Estimators

- **Overview:**

The estimator hierarchy is centred around an abstract *BaseEstimator* class, which defines a minimal contract for all models via the *fit* and *predict* methods. Every concrete estimator must inherit from this class, ensuring that higher-level components such as the *Pipeline* and evaluation utilities can interact with models in a uniform way:

```
class BaseEstimator(ABC):
    @abstractmethod
    def fit(self, X, y=None):
```

```
        """Standardized fit method for all models."""
        pass

    @abstractmethod
    def predict(self, X):
        """Standardized prediction method."""
        pass
```

- **Design choices:**

Supervised and unsupervised models appear as separate branches under this base class. For supervised learning, the *DecisionTreeClassifier* implements a recursive tree structure built from custom *Node* objects. For unsupervised learning, we implemented algorithms of varying complexity to test the robustness of our abstraction. While *KMeans* relies on standard centroid updates, we also integrated spectral clustering, a mathematically intensive algorithm.

The implementation of *SpectralClustering* demonstrates the power of our OOP design by encapsulating complex linear algebra operations behind the standard API. Internally, it computes the affinity matrix using an RBF kernel and derives the normalized Laplacian. It then leverages *numpy.linalg.eigh* to perform *eigendecomposition*, projecting data into a lower-dimensional space before clustering. Despite these internal complexities, both *KMeans* and *SpectralClustering* expose the exact same high-level interface specified by *BaseEstimator*. This design allows users to seamlessly swap a simple geometric model for a graph-theoretic one without modifying the surrounding data handling or preprocessing code.

- **Limitations & improvements:**

The current estimator architecture has some limitations, due to its simplicity and minimalism. For example, there is no built-in support for saving/loading models, automatic hyperparameter validation, or probabilistic predictions (like *predict_proba*). Additionally basic are error messages related to shape mismatches or unfitted models.

Future improvements could include standardised error handling for unfitted estimators, model serialization interface, and an optional *score()* method for quick evaluation. To further match the library with scikit-learn design patterns, mixing classes for specialized behaviours like classifiers, regressors, and clustering models, might be included.

4. Pipeline

- **Overview:**

The *Pipeline* class is at the center of the diagram orchestrating transformers and a final estimator. In order to make the entire workflow appear to the user as a single estimator, it exposes its own *fit* and *predict* methods and internally stores an ordered list of (*name, step*) pairs, where each step is either a *BaseTransformer* or a *BaseEstimator*. The training logic can be summarized as follows:

```python
class Pipeline(BaseEstimator):
    def __init__(self, steps):
        self.steps = steps

    def fit(self, X, y=None):
        X_transformed = X
        for name, step in self.steps[:-1]:
            X_transformed = step.fit_transform(X_transformed, y)
        final_name, final_step = self.steps[-1]
        final_step.fit(X_transformed, y)
        return self

    def predict(self, X):
        X_transformed = X
        for name, step in self.steps[:-1]:
            X_transformed = step.transform(X_transformed)
        final_name, final_step = self.steps[-1]
        return final_step.predict(X_transformed)
```

From the user's point of view, this composition allows an entire chain of preprocessing steps and a model to be treated as a single object, fully consistent with the class-level relationships indicated in the diagram. Any component that implements the expected interface (*fit, transform, fit_transform, or predict*) can be plugged into the pipeline, which reinforces interface consistency and encourages modular experimentation.

- **Design choices:**

A notable design decision was to include an internal step validation method *_validate_steps()*. This checks that there is at least one step, that all but the last step implements both *fit* and *transform*, and that the last step implements *fit*. This prevents runtime errors and keeps the pipeline consistent across different algorithm types. The validation runs at initialization and again when replacing steps through *__setitem__()*. Keeping this logic inside the class avoids confusing errors later and reinforces a strict but user-friendly interface.

- **Limitations & improvements:**

The current pipeline implementation supports the main supervised and unsupervised workflows, but it has a few limitations. It does not yet allow partial fitting or streaming data, and it assumes that everything fits into memory. Additionally, rather than more complex branching graphs, it presupposes a strict linear sequence of steps.

Future improvements could include branching pipelines, automatic type checking between steps, parallel execution for certain transformations and support for caching transformed data. Despite these limitations, the existing implementation already satisfies the fundamental project need of chaining preprocessing and models in a clean, reusable way.

5. Models

- **Overview:**

The models package provides implementations for both supervised and unsupervised learning algorithms. All models inherit from a common *BaseEstimator*, ensuring a consistent interface across library. Each model exposes a *fit()* and *predict()* method (*when applicable*),

making the models easy to use and interchangeable. The design is inspired by established machine learning libraries such as *scikit-learn*.

- **Supervised models:**

Supervised models receive both input features (X) and target labels (y). Two supervised models are implemented: Linear Regression and Decision Tree Classifier. Ordinary least squares are used in Linear Regression to fit a linear function. It enables users to view model parameters directly by supporting an optional intercept term and exposing learnt coefficients after fitting. Decision Tree Classifier builds a decision tree by recursively dividing features based on impurity (Gini or Entropy). It supports depth control and generates class predictions by traversing the tree.

- **Unsupervised models:**

Unsupervised models only take input data (X), since no labels are provided.
Three clustering algorithms are included: K-Means, Agglomerative Clustering, and Spectral Clustering.
K-Means partitions data into clusters by updating centroids until convergence and returns cluster labels and inertia as fitted attributes.
Agglomerative Clustering does hierarchical clustering by iteratively merging samples into clusters following linkage strategy (single, complete, or average). Designed to work through a fit_predict() workflow.
Spectral Clustering builds an affinity matrix, computes a Laplacian, and clusters the resulting embedding using K-Means.

- **Design choices & iterations:**

We adopted inheritance to separate supervised vs unsupervised logic while sharing common estimator behaviour. Models expose attributes similar to *scikit-learn* to make debugging easier. To adhere to the project constraints and make the internal workings of the algorithms clear, implementations were created from scratch using *NumPy*.

- **cluster.py:**

```
class KMeans(UnsupervisedModel):
    def fit(self, X, y=None):
        self.inertia_ = self._calculate_inertia(X)
        return self

class AgglomerativeClustering(UnsupervisedModel):
    def fit(self, X, y=None):
        dist_matrix = self._compute_distance_matrix(X)
        return self

class SpectralClustering(UnsupervisedModel):
    def fit(self, X, y=None):
        laplacian = self._compute_normalized_laplacian(self.affinity_matrix_)
        return self
```

The clustering algorithms in the *cluster.py* file inherit from the *UnsupervisedModel* base class, ensuring a common interface and uniform behaviour among clustering techniques. *K-means, Agglomerative Clustering,* and *Spectral Clustering* are among the techniques used.
All clustering models implements a *fit()* method, which learns the cluster structure from the input data and stores relevant fitted attributes. Since clustering is unsupervised, no target labels are needed.

- **linear_model.py:**

```
class LinearRegression(SupervisedModel):
    def __init__(self, fit_intercept=True):
        self.intercept_ = None

    def fit(self, X, y):
        return self

    def predict(self, X):
        return X @ self.coef_ + self.intercept_
```

The *linear_model.py* file contains *LinearRegression* class. The implementation uses *NumPy* operations to follow a classical least-squares formulation.
Additionally, the model has a *fit(X, y)* and *predict(X)* interfaces. Coefficients and intercept are examples of model parameters that are learnt during training and stored as fitted attributes for later use in prediction. Validation and estimator state checks are made possible by the separation of training from prediction.

- **supervised.py:**

```
class SupervisedModel(BaseEstimator):
    def __init__(self):
        super().__init__()
```

Supervised.py defines the *SupervisedModel* base class, which provides a shared abstraction for all supervised learning algorithms in the library.
By inheriting from a common base class, supervised models enforce a consistent interface and shared behaviour, allowing future supervised models to be added with minimal duplication.

- **tree.py:**

```
class Node:
    def __init__(self, feature=None, threshold=None, left=None, right=None, value=None):
        self.value = value

    def is_leaf(self):
        return self.value is not None

class DecisionTreeClassifier(SupervisedModel):
    def __init__(self, max_depth=None, min_samples_split=2, criterion='gini'):
        self.feature_importances_ = None

    def fit(self, X, y):
        return self

    def predict(self, X):
        return np.array([self._traverse_tree(x, self.tree_) for x in X])

    def _build_tree(self, X, y, depth):
                left=left_child, right=right_child)
```

```
    def _best_split(self, X, y):
        return best_feature, best_threshold

    def _calculate_impurity(self, y):
        return -np.sum(proportions * np.log2(proportions))

    def _most_common_label(self, y):
        return np.bincount(y).argmax()

    def _traverse_tree(self, x, node):
        return self._traverse_tree(x, node.right)
```

Tree.py implements *DecisionTreeClassifier* for supervised classification tasks. The tree is built recursively using a *Node* class to represent both internal decision nodes and leaf nodes.
The classifier learns a hierarchical structure by recursively splitting the dataset based on feature thresholds. Each node stores the information required to traverse the tree during prediction, while leaf nodes store the predicted class value.

- **unsupervised.py:**

```
class UnsupervisedModel(BaseEstimator):
    def __init__(self):
        super().__init__()
```

Unsupervised.py module defines *UnsupervisedModel* base class, which serves as the foundation for clustering and other unsupervised algorithms.
The base class establishes a consistent API for unsupervised models, which require only input data (*X*) during training.

- **Limitations & improvements:**

Advanced features (regularisation for linear models, pruning for trees) are absent from some models. Prediction on new data is not supported by unsupervised models such as *Spectral* and *Agglomerative Clustering*.
Future improvements could be to add regularized linear models, tree pruning, more affinities functions for *Spectral Clustering*, and support for online or partial fitting.

6. Utilities

- **Overview:**

The *utilities* module contains small helper functions that improve consistency and safety across the library. Utilities centralize tasks like input validation and estimator state checking, in order to reduce code repetition and keep the rest of the code cleaner and closer to professional ML libraries.

```
def fit(self, X, y):
    X, y = check_X_y(X, y)
    # training logic ...

def predict(self, X):
    check_is_fitted(self, attributes=["tree_"])
    X = check_array(X)
    # prediction logic ...
```

- **init__.py purpose:**

The *__init__.py* file explains that the *utilities* folder is intended for generic helper functions that do not fit under model selection, metrics, or decorators. It does not contain executable code but clarifies the intention of the folder for contributors.

- **Validation utilities:**

The main logic is in *validation.py*, which handles three key tasks: validating input arrays, checking supervised pairs (*X, y*), and verifying that estimators are fitted before use.
- *check_array*: This function converts input into *NumPy* array and rejects None, empty arrays, or 0-D data. Ensuring models receive clean and consistent numeric inputs.
- *check_X_y*: This function validates supervised pairs (*X, y*) pairs by running *check_array* on *X*, converting *y* to *NumPy* array, and checking that both have matching sample sizes.
- *check_is_fitted*: This function verifies that an estimator has been trained by checking for fitted attributes and raises a clear error if the model has not been fitted.

- **Design choices:**

Validation was separated from models to avoid repeated checks across library. Using *NumPy* ensures numerical consistency, and clear error messages help users understand and correct mistakes.

- **Limitations & future extensions:**

Only basic array and estimator tests are covered by current validation. Type checking, sparse data handling, missing value detection, and decorators-based automatic validation are some potential future enhancements.

IV) Evaluation of the Library and Comparative Analysis with PyTorch

- **Evaluation Strategy:**

To validate *upsaclay_ml,* we employed a dual evaluation strategy combining unit testing and comparative benchmarking. Unit tests focus on mathematical correctness, checking that low-level operations and algorithmic components behave as expected under controlled conditions and for edge cases such as empty data sets or extreme hyper parameter values. Comparative benchmarks assess performance by comparing our implementations with established libraries on practical tasks, using shared hyper parameters and random seeds to ensure fair comparisons.

The objective is to show that although a from-scratch Python implementation cannot match the raw speed of C-optimized libraries, it preserves the same theoretical behaviour and numerical results. Together, these two dimensions correctness and performance provide a balanced view of the library and help answer two central questions: *"Does the implementation behave as the theory predicts?"* and *"Is its runtime acceptable for typical educational use cases?"*.

- **Performance Benchmark: Computation Time**

We compared the training latency of our *KMeans* implementation against the industry-standard *scikit-learn* library. The benchmark measured execution time across datasets ranging from *N = 100* to *N = 5,000* samples on a synthetic *Blobs* dataset, and the results are summarized in *Figure 2*. For each dataset size, both implementations were trained with the same number of clusters, maximum iterations, and random seed, and all experiments were run on the same machine to avoid hardware-related biases.

A simplified version of the benchmarking loop is shown below:

```
sizes = [100, 500, 1000, 5000]
times_ours = []
times_sklearn = []

for n in sizes:
    X, _ = make_blobs(n_samples=n, n_features=2, random_state=0)

    start = time.time()
    kmeans_ours.fit(X)
    times_ours.append(time.time() - start)

    start = time.time()
    kmeans_sklearn.fit(X)
    times_sklearn.append(time.time() - start)
```

This snippet illustrates how training time is recorded for both implementations as the dataset size increases, using identical experimental conditions.

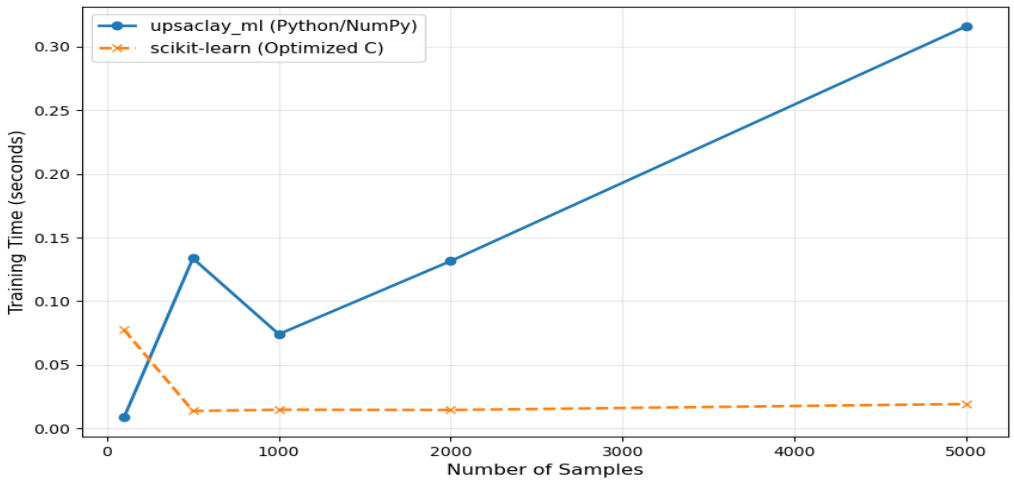


Figure 2: Training time comparison between upsaclay_ml (Blue) and scikit-learn (Orange) on the Blobs dataset.

The *figure 2* shows a distinct performance gap between the two libraries. *Scikit-learn* (orange curve) displays near-constant low latency (around 0.02 s), which is expected given that its core algorithms are implemented in *Cython* and highly optimized C, leveraging low-level memory management and parallelization.

From an observation we can note that *upsaclay_ml* (blue curve) exhibits a roughly linear increase in execution time, reaching approximately 0.32 s for 5,000 samples, which indicates that the implementation follows the expected asymptotic complexity O(T·K·N·D) rather than suffering from hidden bottlenecks. In the context of a teaching-oriented library, this overhead is acceptable, since training times remain well below one second even for several thousand samples, making the implementation suitable for classroom demonstrations and small-scale experiments.

4.3 Accuracy Analysis: Confusion Matrix

To evaluate predictive power, we trained our *DecisionTreeClassifier* on the *Iris* dataset, using 105 samples for training and 45 for testing.

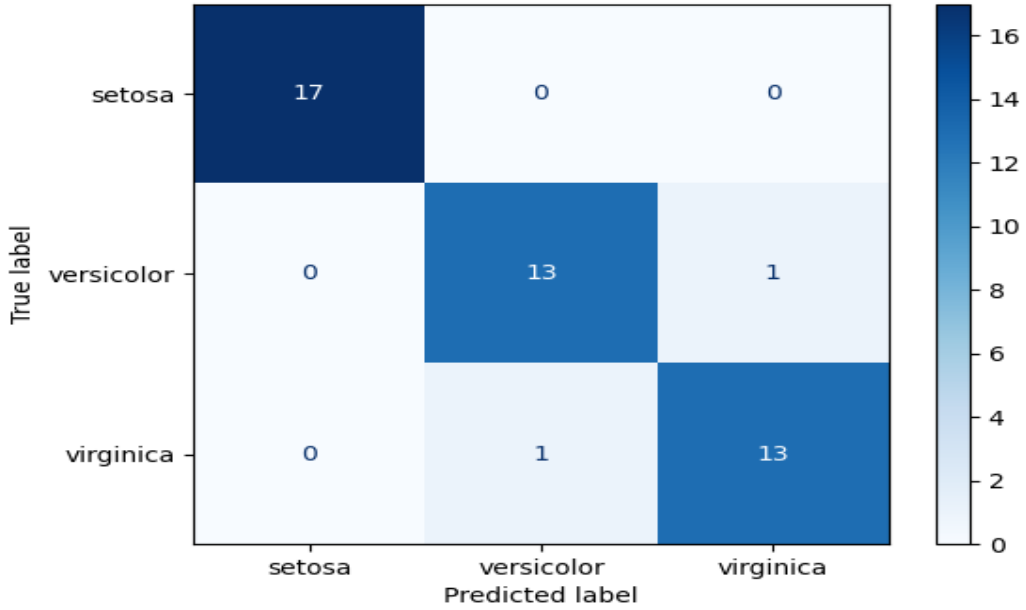


Figure 3: Confusion Matrix of the Decision Tree Classifier evaluated on the Iris Test Set.

A confusion matrix (*Figure 3*) was used to visualize classification performance across the three classes and to inspect the distribution of errors. The following code snippet shows how the confusion matrix is computed and displayed:

This snippet of script below highlights how the evaluation pipeline integrates training, prediction, and diagnostic visualization for the custom decision tree implementation. The

model achieved a global accuracy of 95.6% (43 correct predictions out of 45). The *Setosa* class was perfectly classified (17/17), which is consistent with classical analyses of the dataset, since *Setosa* is linearly separable from the other two classes. The matrix reveals slight confusion between *Versicolor* and *Virginica*, with one sample of each class misclassified as the other, reflecting the known overlap of their feature distributions in sepal and petal dimensions.

```
tree.fit(X_train, y_train)
y_pred = tree.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(cm, display_labels=tree.classes_)
disp.plot()
```

The fact that these errors mirror the typical behavior of standard decision tree implementations on this dataset confirms that our implementations of *Gini Impurity* and *Information Gain* are mathematically precise. Combined with the timing results from *KMeans*, this accuracy analysis suggests that *upsaclay_ml* reaches a good compromise between faithfulness to theory and readability of the underlying code.

V) Limits of our model and implementation:

1. Dense vs. Sparse Matrix Implementation:

We currently represent all datasets and distance matrices using standard dense *NumPy* arrays. This requires storing an *NxN* affinity matrix in memory with $O(N^2)$ complexity for algorithms such as *Spectral Clustering*. *Scikit-learn* significantly reduces memory footprint by handling datasets with majority of entries being zero, utilising *scipy.sparse* (CSR/CSC formats). Our library could scale beyond 20,000 samples without causing *MemoryError* if sparse support was implemented.

2. Interpreter Overhead vs. Compiled Extensions:

Our library operates entirely in the Python user space. *NumPy* optimizes vector operations, but iterative algorithms (like the main loop in K-Means or tree recursion) incur the overhead of the Python Global Interpreter Lock (GIL). By implementing critical solvers in *Cython* (C-extensions), *Scikit-learn* bypasses this and allows machine-level optimization and direct memory access. To match industrial training speeds, we would need to port our *DecisionTree* recursion to *Cython*.

3. Strategies for Initialization:

Presently, we can either employ a reduced K-Means++ implementation or conventional random selection to initialize K-Means centroids. Highly optimized initialization approaches that compute first candidate distances in parallel are incorporated into Scikit-

learn. Convergence stability would be improved by making our initialization logic more resilient to local minima.

4. Parallelization and CPU Utilization:

Currently, our library runs on a single CPU core (*single-threaded*). When performing computationally costly operations (*such as distance calculations*), *Scikit-learn* makes use of all available CPU cores by utilising OpenMP functions within its C-backend. Our library may become much more competitive for large-scale studies by adding parallel processing capabilities (*via multiprocessing or joblib*).

5. Hyperparameter Tuning and Validation:

While we provide a basic train/test split, our library does not have an automated hyperparameter optimisation technique. *Scikit-learn* provides *GridSearchCV* and *CrossValScore* to methodically test parameter combinations (*tree depth, number of clusters*). In order to automate model selection, future development should concentrate on creating a generic *SearchCV* class that inherits from *BaseEstimator*.

VI) Conclusion

This project resulted in the development of a machine learning library built from scratch using *Python* and *object-oriented* design principles. By avoiding existing *ML* implementations, the work demonstrates how core algorithms, data processing tools, and evaluation methods operate under the hood, while also highlighting the importance of software structure, modularity, and clean interfaces.

Although the library cannot match optimized frameworks such as *scikit-learn* in computational speed or feature breadth, it provides a functional and readable end-to-end workflow that includes data loading, preprocessing, model training, and evaluation. Clear architectural choices, consistent interfaces, and transparent code help make the library educational and accessible. Limitations and possible improvements have also been identified, indicating several opportunities for future extensions.

Overall, this project shows that building a machine learning pipeline from first principles offers a valuable understanding of algorithmic mechanics and software design and provides a solid foundation for further development.